importing the needed libraries

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import MinMaxScaler
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score,precision_score, recall_sc
from sklearn.metrics import confusion_matrix
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
import numpy as np
```

importing the dataset and exploring it

```
df = pd.read_csv('/content/survey lung cancer (2).csv')
```

```
df.head()
```

	GENDER	AGE	SMOKING	YELLOW_FINGERS	ANXIETY	PEER_PRESSURE	CHRONIC DISEASE
0	M	69	1	2	2	1	1
1	M	74	2	1	1	1	2
2	F	59	1	1	1	2	1
3	M	63	2	2	2	1	1
4	F	63	1	2	1	1	1

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 309 entries, 0 to 308
Data columns (total 16 columns):
 #   Column                 Non-Null Count  Dtype
---  ------                 --------------  -----
 0   GENDER                 309 non-null    object
 1   AGE                    309 non-null    int64
 2   SMOKING                309 non-null    int64
 3   YELLOW_FINGERS         309 non-null    int64
 4   ANXIETY                309 non-null    int64
 5   PEER_PRESSURE          309 non-null    int64
 6   CHRONIC DISEASE        309 non-null    int64
 7   FATIGUE                309 non-null    int64
```

```
 8   ALLERGY                 309 non-null    int64
 9   WHEEZING                309 non-null    int64
 10  ALCOHOL CONSUMING       309 non-null    int64
 11  COUGHING                309 non-null    int64
 12  SHORTNESS OF BREATH     309 non-null    int64
 13  SWALLOWING DIFFICULTY   309 non-null    int64
 14  CHEST PAIN              309 non-null    int64
 15  LUNG_CANCER             309 non-null    object
dtypes: int64(14), object(2)
memory usage: 38.8+ KB
```

```
df.isnull().sum()
```

```
GENDER                  0
AGE                     0
SMOKING                 0
YELLOW_FINGERS          0
ANXIETY                 0
PEER_PRESSURE           0
CHRONIC DISEASE         0
FATIGUE                 0
ALLERGY                 0
WHEEZING                0
ALCOHOL CONSUMING       0
COUGHING                0
SHORTNESS OF BREATH     0
SWALLOWING DIFFICULTY   0
CHEST PAIN              0
LUNG_CANCER             0
dtype: int64
```

```
df.duplicated().sum()
```

```
33
```

preprocessing

encoding

```
# I encoded all the columns to convert the categorical columns to num
for col in df.columns:
    df[col] = LabelEncoder().fit_transform(df[col])
```

normlization

```
df['AGE'] = MinMaxScaler().fit_transform(df[['AGE']])
```

dropping duplicates

```
df.drop_duplicates(inplace=True)
```

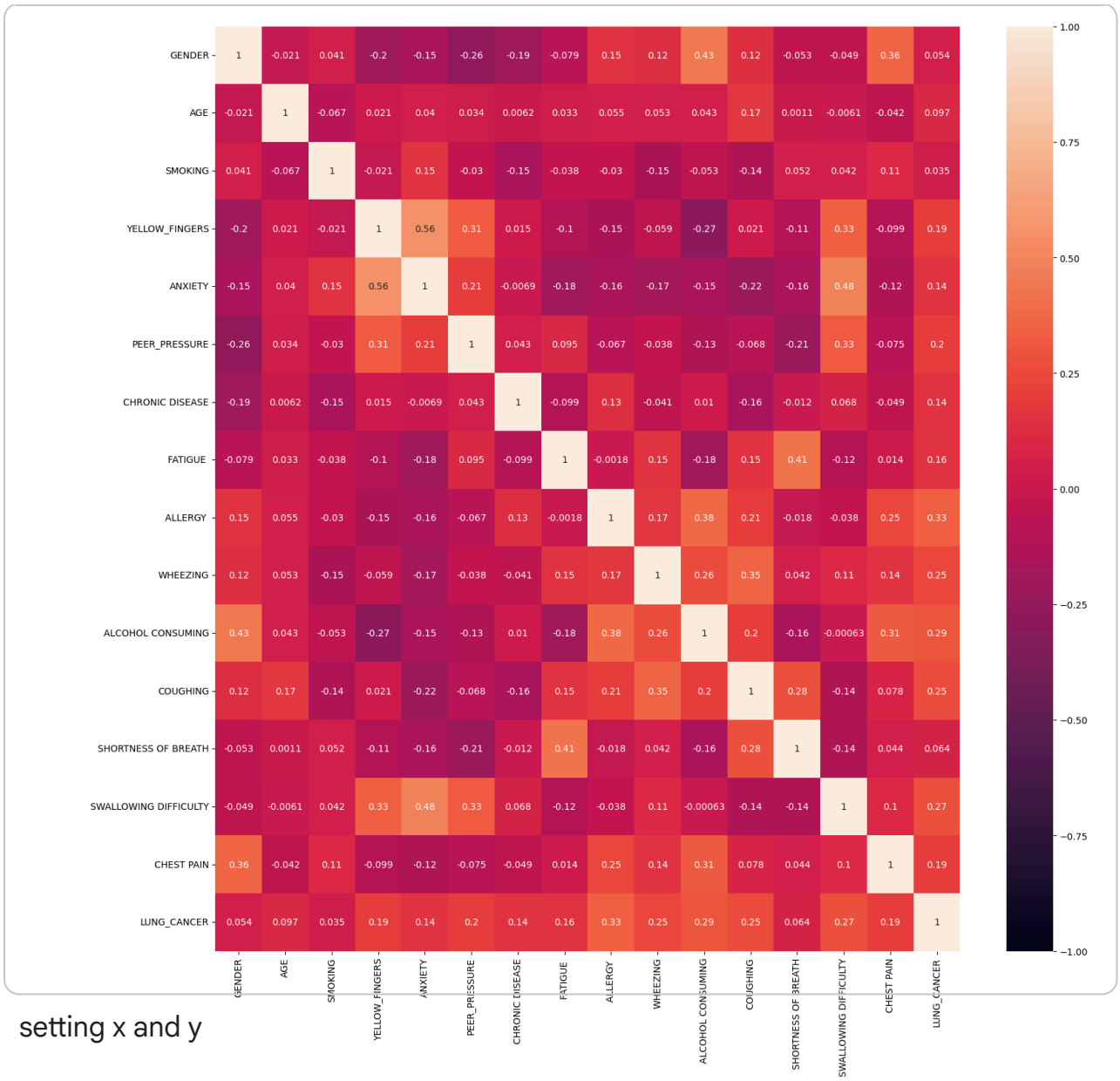
dataframe after normlization and encoding

```
df.head()
```

	GENDER	AGE	SMOKING	YELLOW_FINGERS	ANXIETY	PEER_PRESSURE	CHR DIS
0	1	0.684211	0	1	1	0	
1	1	0.815789	1	0	0	0	
2	0	0.421053	0	0	0	1	
3	1	0.526316	1	1	1	0	
4	0	0.526316	0	1	0	0	

corelation between features

```
plt.figure(figsize=(18, 18))
sns.heatmap(df.corr(), annot=True, vmin = -1,vmax=1)
plt.show()
```



setting x and y

```
X = df.drop(['LUNG_CANCER'],axis=1)
y = df['LUNG_CANCER']
```

KNN classification model

```
X_train, X_test, y_train, y_test_knn = train_test_split(X, y, test_si

kNN_class = KNeighborsClassifier(n_neighbors=3)

kNN_class.fit(X_train, y_train)

knn_y_predict = kNN_class.predict(X_test)
```

evaluation

```
knn_accuracy = accuracy_score(y_test_knn, knn_y_predict)
knn_precision = precision_score(y_test_knn, knn_y_predict)
knn_recall = recall_score(y_test_knn, knn_y_predict)
knn_f1 = f1_score(y_test_knn, knn_y_predict)
print(f'The Accuracy of KNN: {knn_accuracy}')
print(f'The precision of KNN: {knn_precision}')
print(f'The recall of KNN: {knn_recall}')
print(f'The F1 score KNN: {knn_f1}')
```

```
The Accuracy of KNN: 0.9036144578313253
The precision of KNN: 0.9444444444444444
The recall of KNN: 0.9444444444444444
The F1 score KNN: 0.9444444444444444
```

confusion_matrix

```
confusion_matrix(y_test_knn, knn_y_predict)
```

```
array([[ 7,  4],
       [ 4, 68]])
```

Decision tree classifier

```
X_train, X_test, y_train, y_test_dt = train_test_split(X, y, test_siz

dt_class = DecisionTreeClassifier()

dt_class.fit(X_train, y_train)

dt_y_predict = dt_class.predict(X_test)
```

evaluation

```
dt_accuracy = accuracy_score(y_test_dt, dt_y_predict)
dt_precision = precision_score(y_test_dt, dt_y_predict)
dt_recall = recall_score(y_test_dt, dt_y_predict)
dt_f1 = f1_score(y_test_dt, dt_y_predict)

print(f'The Accuracy of Decision Tree: {dt_accuracy}')
print(f'The precision of Decision Tree: {dt_precision}')
print(f'The recall of Decision Tree: {dt_recall}')
print(f'The F1 score Decision Tree: {dt_f1}')
```

```
The Accuracy of Decision Tree: 0.891566265060241
The precision of Decision Tree: 0.9436619718309859
The recall of Decision Tree: 0.9305555555555556
The F1 score Decision Tree: 0.9370629370629372
```

confusion_matrix

```
confusion_matrix(y_test_dt, dt_y_predict)
```

```
array([[ 7,  4],
       [ 5, 67]])
```

Random forest classifier

```
X_train, X_test_rf, y_train, y_test_rf = train_test_split(X, y, test_

rf = RandomForestClassifier()

rf.fit(X_train, y_train)

rf_y_predict = rf.predict(X_test)
```

evaluation

```
rf_accuracy = accuracy_score(y_test_rf, rf_y_predict)
rf_precision = precision_score(y_test_rf, rf_y_predict)
rf_recall = recall_score(y_test_rf, rf_y_predict)
rf_f1 = f1_score(y_test_rf, rf_y_predict)

print(f'The Accuracy of Decision Tree: {rf_accuracy}')
print(f'The precision of Decision Tree: {rf_precision}')
print(f'The recall of Decision Tree: {rf_recall}')
print(f'The F1 score Decision Tree: {rf_f1}')
```

```
The Accuracy of Decision Tree: 0.927710843373494
The precision of Decision Tree: 0.9583333333333334
The recall of Decision Tree: 0.9583333333333334
The F1 score Decision Tree: 0.9583333333333334
```

confusion_matrix

```
confusion_matrix(y_test_rf, rf_y_predict)
```

```
array([[ 8,  3],
       [ 3, 69]])
```

visualization and analysis

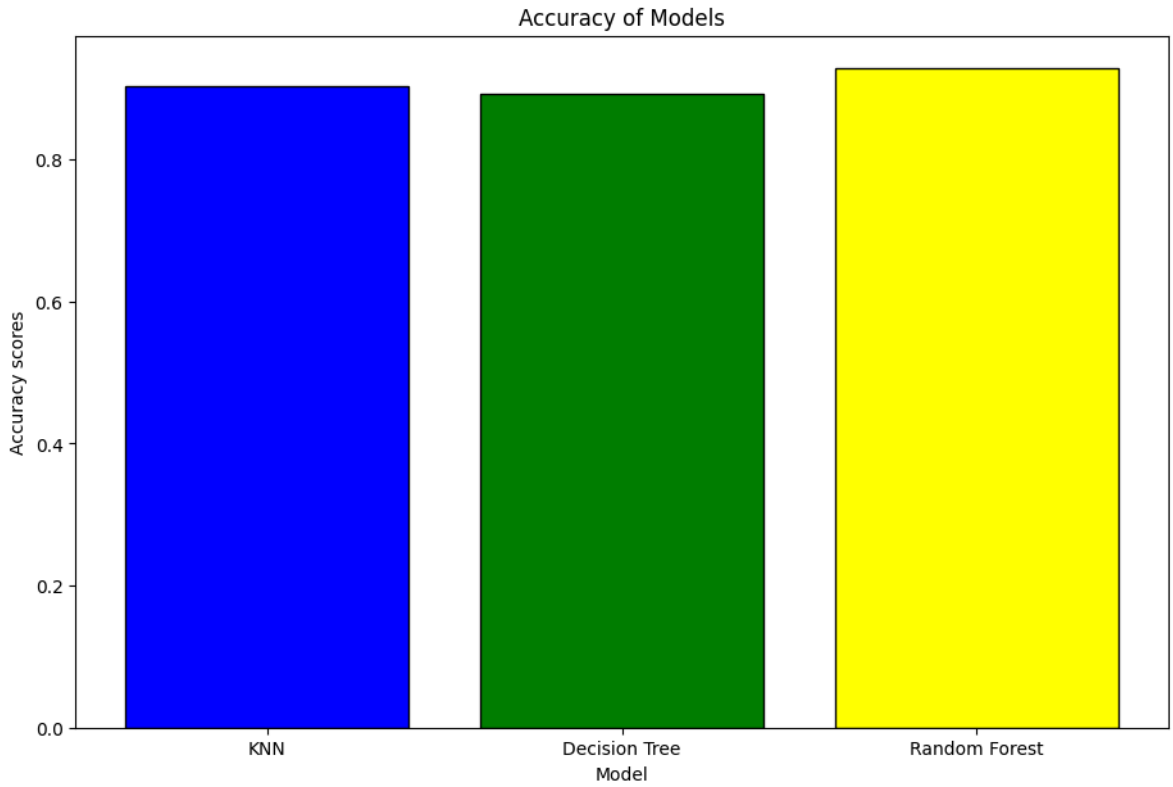
```
models = ['KNN', 'Decision Tree', 'Random Forest']
```

```
color = ['blue', 'green', 'yellow']
```

accuracy

```
accuracy_score = [knn_accuracy,dt_accuracy,rf_accuracy]
```

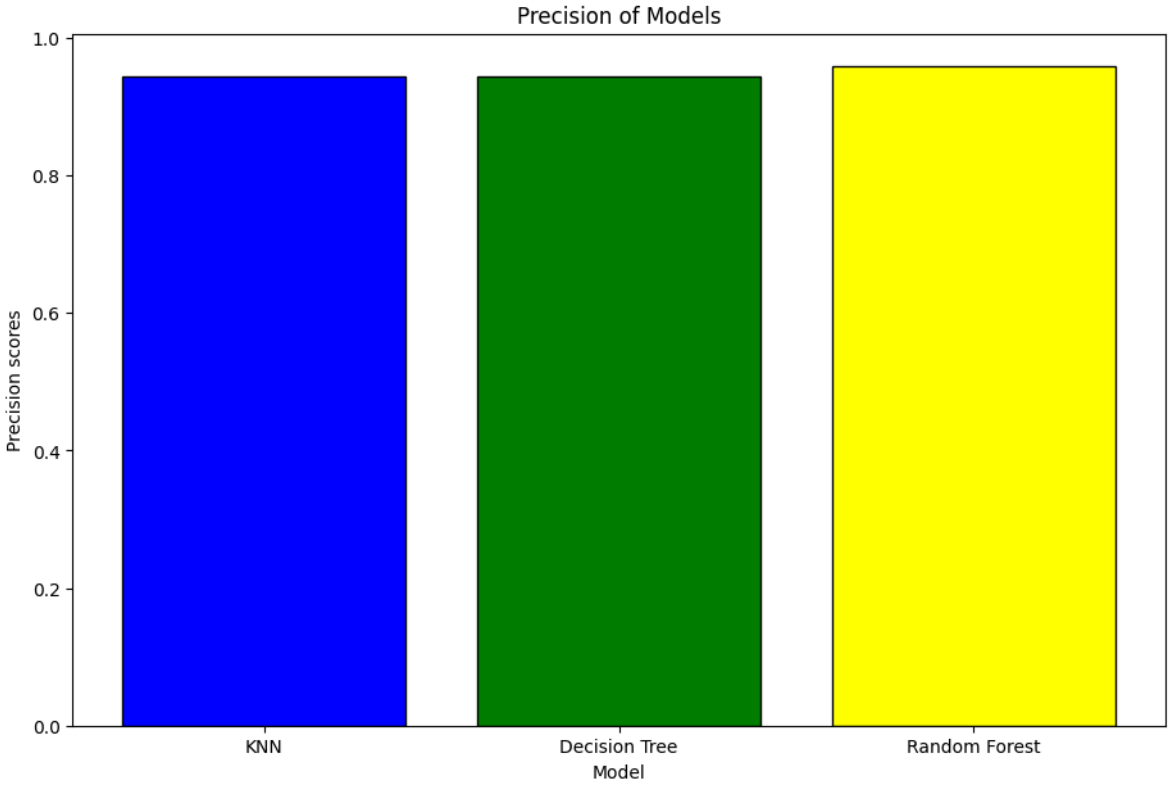
```
plt.figure(figsize=(11, 7))
plt.bar(models ,accuracy_score , color=color, edgecolor='black')
plt.title('Accuracy of Models')
plt.ylabel('Accuracy scores')
plt.xlabel('Model')
plt.show()
```



precision

```
precision_score = [knn_precision,dt_precision,rf_precision]
```

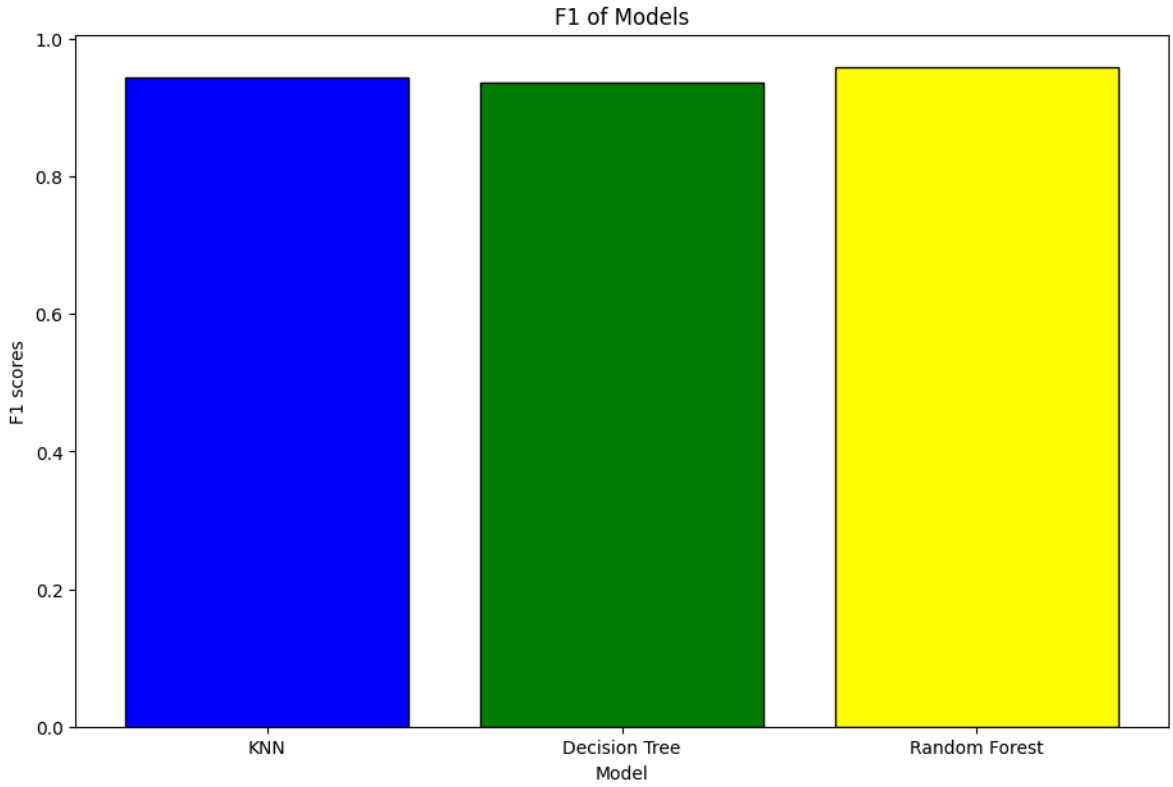
```
plt.figure(figsize=(11, 7))
plt.bar(models , precision_score, color=color, edgecolor='black')
plt.title('Precision of Models')
plt.xlabel('Model')
plt.ylabel('Precision scores')
plt.show()
```



F1

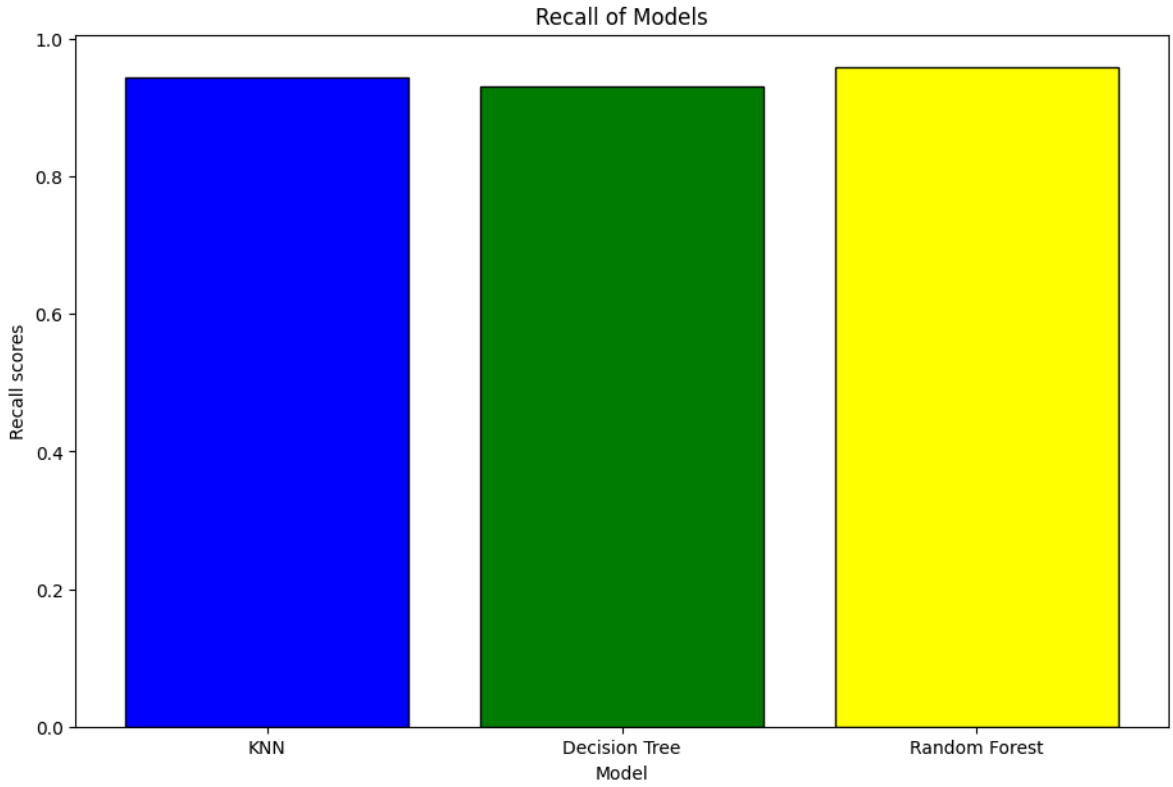
```
f1_score = [knn_f1,dt_f1,rf_f1]
```

```
plt.figure(figsize=(11, 7))
plt.bar(models , f1_score, color=color, edgecolor='black')
plt.title('F1 of Models')
plt.xlabel('Model')
plt.ylabel('F1 scores')
plt.show()
```

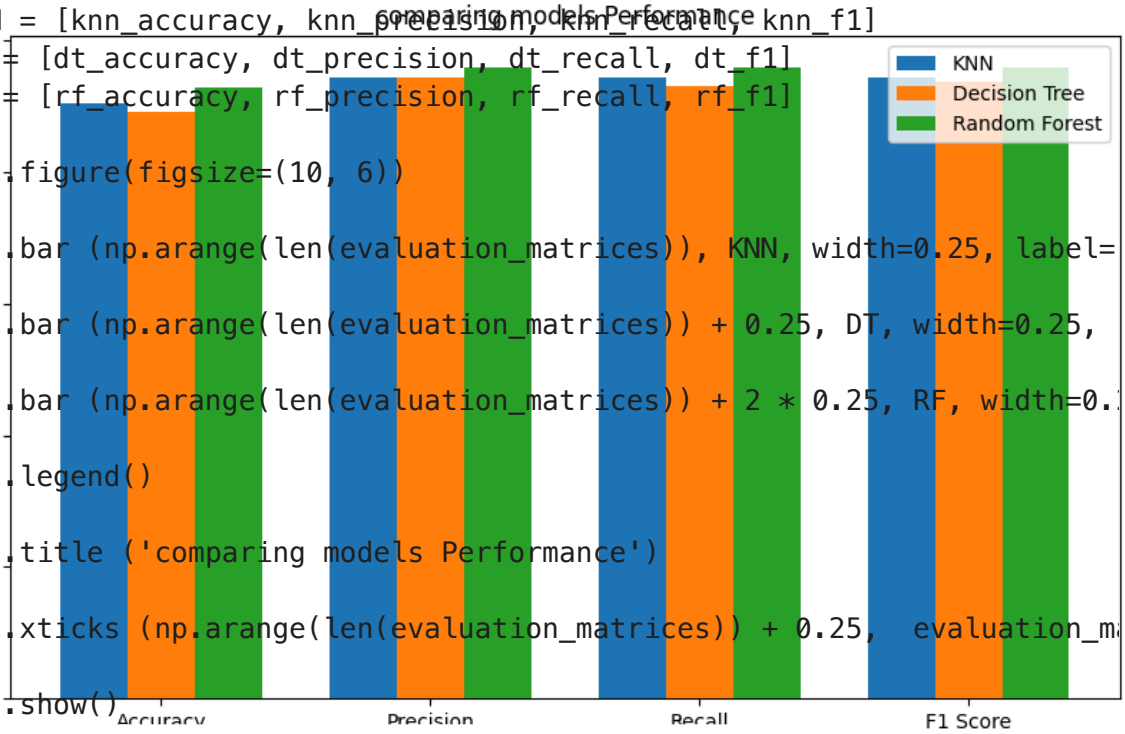


recall

```
recall_score = [knn_recall,dt_recall,rf_recall]
plt.figure(figsize=(11, 7))
plt.bar(models , recall_score, color=color, edgecolor='black')
plt.title('Recall of Models')
plt.xlabel('Model')
plt.ylabel('Recall scores')
plt.show()
```


Recall of Models
1.0
0.8
0.6
0.4
0.2
0.0
Recall scores
KNN
Decision Tree
Random Forest
Model

```
evaluation_matrices = ['Accuracy', 'Precision', 'Recall', 'F1 Score']
KNN = [knn_accuracy, knn_precision, knn_recall, knn_f1]
DT = [dt_accuracy, dt_precision, dt_recall, dt_f1]
RF = [rf_accuracy, rf_precision, rf_recall, rf_f1]

plt.figure(figsize=(10, 6))

plt.bar (np.arange(len(evaluation_matrices)), KNN, width=0.25, label=

plt.bar (np.arange(len(evaluation_matrices)) + 0.25, DT, width=0.25,

plt.bar (np.arange(len(evaluation_matrices)) + 2 * 0.25, RF, width=0.

plt.legend()

plt.title ('comparing models Performance')

plt.xticks (np.arange(len(evaluation_matrices)) + 0.25,  evaluation_m

plt.show()
```



comparing the scores

```
print(f'The precision of KNN: {knn_precision}')
print(f'The precision of Decision Tree: {dt_precision}')
print(f'The precision of Random Forest: {rf_precision}')
```

```
The precision of KNN: 0.9444444444444444
The precision of Decision Tree: 0.9436619718309859
The precision of Random Forest: 0.9583333333333334
```

```
print(f'The F1 score KNN: {knn_f1}')
```